

Django Nesneleri Serileştirmek ¶

Django'nun serileştirme çerçevesi, Django modellerini başka biçimlere “dönüştürmek” için bir mekanizma sunar.



See also

If you just want to get some data from your tables into a serialized form, you could use the [dumpdata](#) management command.

Verilerin Sıralanması ¶

En üst düzeyde, verileri şu şekilde serileştirebilirsiniz:

```
from django.core import serializers

data = serializers.serialize("json", SomeModel.objects.all())
```

Serialize fonksiyonunun argümanları; verinin hangi biçimde serileştirileceği (bkz. Serileştirme formatları) ve serileştirilecek olan **QuerySet**'tir. (Aslında ikinci argüman, Django model örnekleri üreten herhangi bir "iterator" olabilir; ancak bu neredeyse her zaman bir QuerySet olacaktır.)

django.core.serializers.get_serializer(*format*) ¶

Ayrıca Serileyici Öğeleri direkt kullanabilirsiniz.

```
JSONSerializer = serializers.get_serializer("json")
json_serializer = JSONSerializer()
json_serializer.serialize(queryset)
data = json_serializer.getvalue()
```

Bu yöntem verileri doğrudan dosya benzeri bir nesneye (**HttpResponse** dahil) serileştirmek istediğinizde kullanışlıdır:

```
with open("file.json", "w") as out:
    json_serializer.serialize(SomeModel.objects.all(), stream=out)
```



Note

Calling [get_serializer\(\)](#) with an unknown *format* will raise a **django.core.serializers.SerializerDoesNotExist** exception.

Alanların Alt Kümesi ¶

Yalnızca belirli bir alt kümedeki alanların serileştirilmesini istiyorsanız, serileştiriciye bir **fields**(alan,sınıf) argümanı belirtebilirsiniz:

```
from django.core import serializers

data = serializers.serialize("json", SomeModel.objects.all(), fields=["name", "size"])
```

Bu örnekte, her modelin yalnızca **ismi** ve **boyutu**, öznitelikleri serileştirilecektir. Birincil anahtar, sonuç çıktısında her zaman **pk** ögesi olarak serileştirilir; **fields**(alan,sınıf) bölümünde asla görünmez.



Note

Depending on your model, you may find that it is not possible to deserialize a model that only serializes a subset of its fields. If a serialized object doesn't specify all the fields that are required by a model, the deserializer will not be able to save deserialized instances.

Kalıtılabilir Modeller ¶

Eğer **soyut bir temel** sınıf kullanılarak tanımlanmış bir modeliniz varsa, bu modeli serileştirmek için özel bir işlem yapmanız gerekmez. Seri hale getirmek istediğiniz nesne (veya nesneler) üzerinde seri hale getirme fonksiyonunu çağırın; çıktı, seri hale getirilmiş nesnenin tam bir temsiliyi içerecektir. Ancak, **çoklu tablo mirasını** kullanan bir modeliniz varsa, modelin tüm temel sınıflarını da serileştirmeniz gerekir. Bunun nedeni, yalnızca model üzerinde yerel olarak tanımlanan alanların serileştirilecek olmasıdır. Örneğin, aşağıdaki modelleri ele alalım:

```
class Place(models.Model):
    name = models.CharField(max_length=50)

class Restaurant(Place):
    serves_hot_dogs = models.BooleanField(default=False)
```

Yalnızca Restaurant modelini serileştirirseniz:

```
data = serializers.serialize("json", Restaurant.objects.all())
```

Seri hale getirilmiş çıktıda yer alan alanlar yalnızca “`serves_hot_dogs`” özniteliğini içerecektir. Ana sınıfın “`İsim`” özniteliği göz ardı edilecektir.

“`Restaurant`” nesnelerinizi tam olarak seri hale getirmek için, “`Place`” modellerini de seri hale getirmeniz gerekecektir

```
all_objects = [*Restaurant.objects.all(), *Place.objects.all()]
data = serializers.serialize("json", all_objects)
```